Cloud Infrastructure and DevOps Engineer

URL Shortener using Lambda + DynamoDB

End-to-End Project Report

Student Name: Akshith Reddy Ponna

Roll Number: 2451-23-751-028

Domain: Cloud Infrastructure and DevOps Engineer

Title: URL Shortener using Lambda + DynamoDB

GitHub: https://github.com/Akshith7707/URL-shortner

Region (designed): ap-south-1 (Mumbai)

IaC: AWS CDK (TypeScript)

Date: 31 October 2026

This report documents architecture, implementation, IAM design, local verification with screenshots, and future cloud deployment notes.

1. Introduction and Objective

The goal of this project is to design and implement a serverless URL shortener on AWS using Amazon API Gateway, AWS Lambda, and Amazon DynamoDB, managed with Infrastructure as Code (AWS CDK). The system accepts a long URL, returns a short code, and redirects users from the short link to the original URL.

Key learning outcomes covered:

- Serverless architecture design on AWS
- Infrastructure as Code with AWS CDK (TypeScript)
- Least-privilege IAM for Lambda and DynamoDB
- API design for create and redirect flows
- CloudWatch logging and monitoring design
- Local zero-cost validation using DynamoDB Local

2. Problem Statement

Long URLs are difficult to share. A URL shortener stores a mapping between a compact short code and the original URL. When a client requests the short code, the service responds with an HTTP 301 redirect to the long URL. The solution must be scalable, low-maintenance, and cost-efficient. A serverless design fits these requirements.

3. System Architecture

The application is fully serverless. Clients call an HTTP API. API Gateway routes requests to Lambda functions. Lambda reads and writes mappings in DynamoDB. CloudWatch collects logs and error alarms.

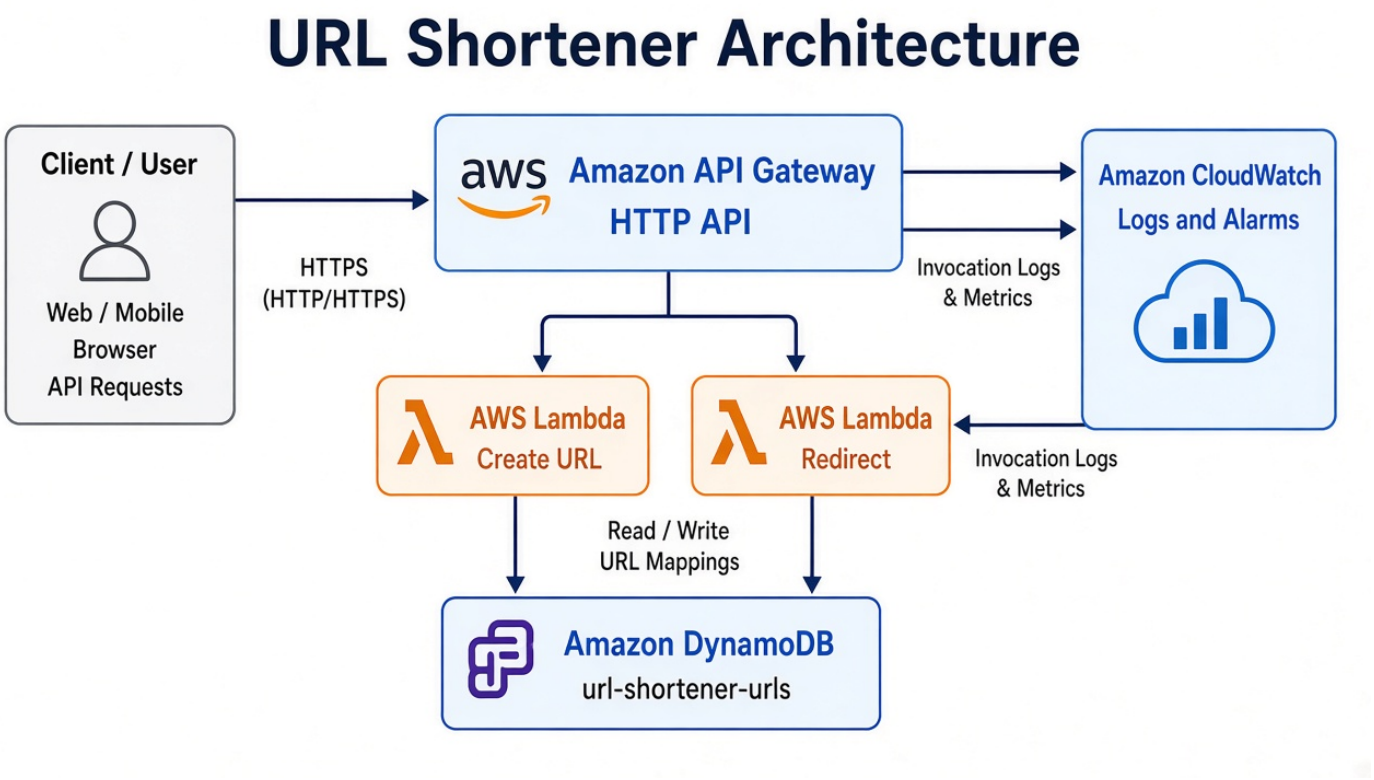


Figure 1. High-level architecture (Client -> API Gateway -> Lambda -> DynamoDB + CloudWatch).

Request flows:

- POST /shorten: Create Lambda validates URL, generates a base62 short code, PutItem into DynamoDB, returns shortUrl.
- GET /{code}: Redirect Lambda GetItem by shortCode; on hit returns 301 Location; increments hitCount.
- GET /health: Liveness check returning { status: ok }.
- Unknown code: HTTP 404 Short URL not found.

4. AWS Services Used

Amazon API Gateway (HTTP API)

Public HTTPS front door; routes /shorten, /health, /{code}; CORS and access logs.

AWS Lambda (Node.js 24)

Business logic for create and redirect handlers; auto-scales; pay per invoke.

Amazon DynamoDB

Key-value store for shortCode -> longUrl; on-demand billing mode in CDK stack.

IAM

Least-privilege execution roles. Create: PutItem only. Redirect: GetItem + UpdateItem only.

Amazon CloudWatch

Lambda log groups, API access logs (7-day retention), error alarms.

AWS CDK / CloudFormation

Defines and synthesizes all infrastructure as code for repeatable deployments.

AWS X-Ray (configured)

Tracing.ACTIVE enabled on Lambda for distributed tracing when deployed.

5. Data Model

Table name: url-shortener-urls

- Partition key: shortCode (String)
- longUrl (String) - destination URL
- createdAt (String) - ISO-8601 timestamp
- hitCount (Number) - redirect counter
- lastAccessedAt (String) - last successful redirect time

Short codes are 7-character base62 values generated with crypto.randomBytes. Writes use ConditionExpression attribute_not_exists(shortCode) with retry on collision.

6. Implementation Overview

Project structure:

- bin/url-shortener.ts - CDK app entry (region ap-south-1)
- lib/url-shortener-stack.ts - DynamoDB, Lambdas, HTTP API, IAM, Logs, Alarms
- lambda/create-url/index.ts - POST /shorten + GET /health
- lambda/redirect/index.ts - GET /{code} redirect
- lambda/shared/dynamo.ts - DynamoDB client with local endpoint support
- scripts/local-api.ts - Local API Gateway stand-in
- scripts/local-setup*.ps1 - DynamoDB Local setup (Docker or JAR)
- docs/ - architecture, local guide, this report

7. API Contract

- GET /health -> 200 { status: ok }
- POST /shorten body { "url": "https://..." } -> 201 { shortCode, shortUrl, longUrl, createdAt }
- GET /{code} -> 301 Location: <longUrl>
- Invalid/missing URL -> 400; unknown code -> 404

8. IAM Least Privilege

- Create Lambda role: dynamodb:PutItem on table ARN only (+ basic logs + X-Ray)
- Redirect Lambda role: dynamodb:GetItem and dynamodb:UpdateItem only
- API Gateway invoke permissions scoped to routes via Lambda resource policy
- No AWS credentials hardcoded in source code

9. Monitoring Design

- Log groups: /aws/lambda/url-shortener-create, /aws/lambda/url-shortener-redirect
- API access logs: /aws/apigateway/url-shortener
- Alarms: url-shortener-create-errors, url-shortener-redirect-errors (Errors >= 1)
- Retention: 7 days for demo cost control

10. Testing and Verification

To keep AWS spend at zero during development and evaluation, the stack was synthesized with CDK and executed end-to-end locally using DynamoDB Local and a local HTTP API that invokes the same Lambda handlers. Cloud deploy (cdk bootstrap / cdk deploy) was intentionally not performed with the root user.

Automated tests:

- CDK unit tests (Jest assertions on DynamoDB, Lambda, HTTP API, IAM, outputs) - passed
- Local handler integration tests against DynamoDB Local - 3/3 passed
- Smoke script scripts/smoke-test.ps1 - health, create, 301 redirect, 404 - passed



Figure 2. Health endpoint response in browser: GET http://127.0.0.1:3000/health => {"status":"ok"}.

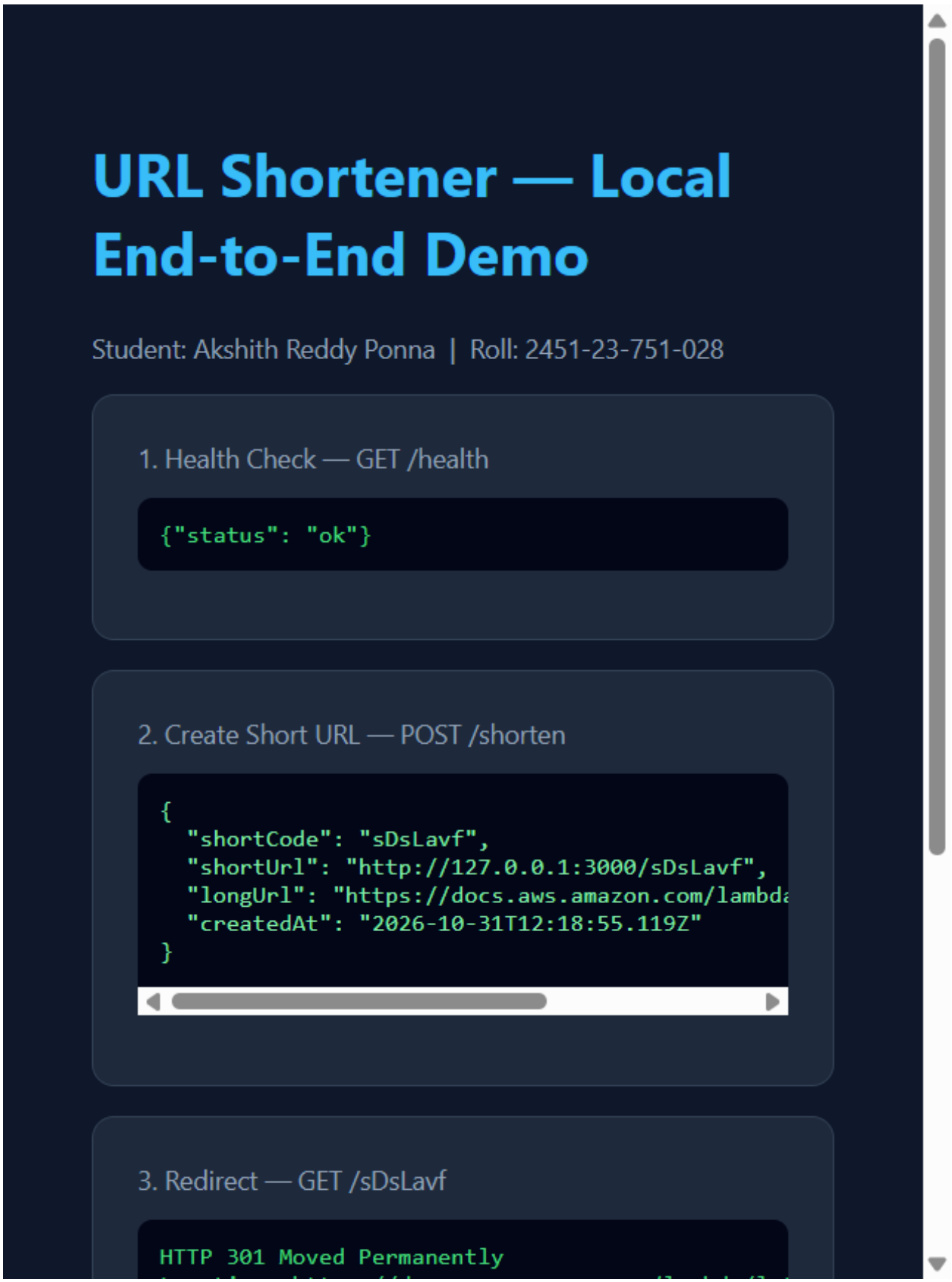


Figure 3. Local end-to-end demo page showing health check and successful POST /shorten response with shortCode.

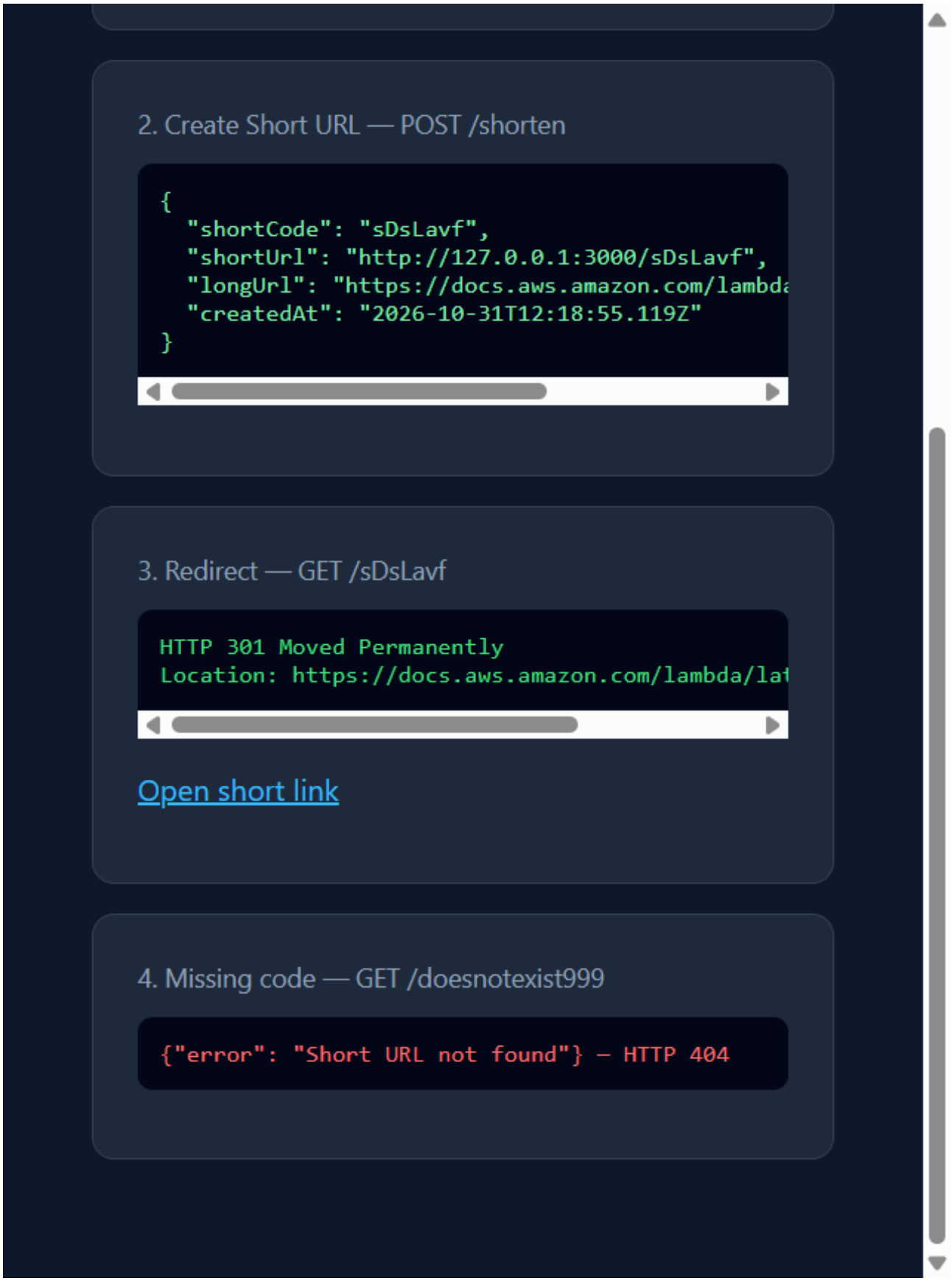


Figure 4. Redirect success (HTTP 301 + Location) and missing-code handling (HTTP 404).

11. Repository and Submission Artifact

Source code, infrastructure, documentation, and local tooling are published on GitHub.

Repository: https://github.com/Akshith7707/URL-shortner

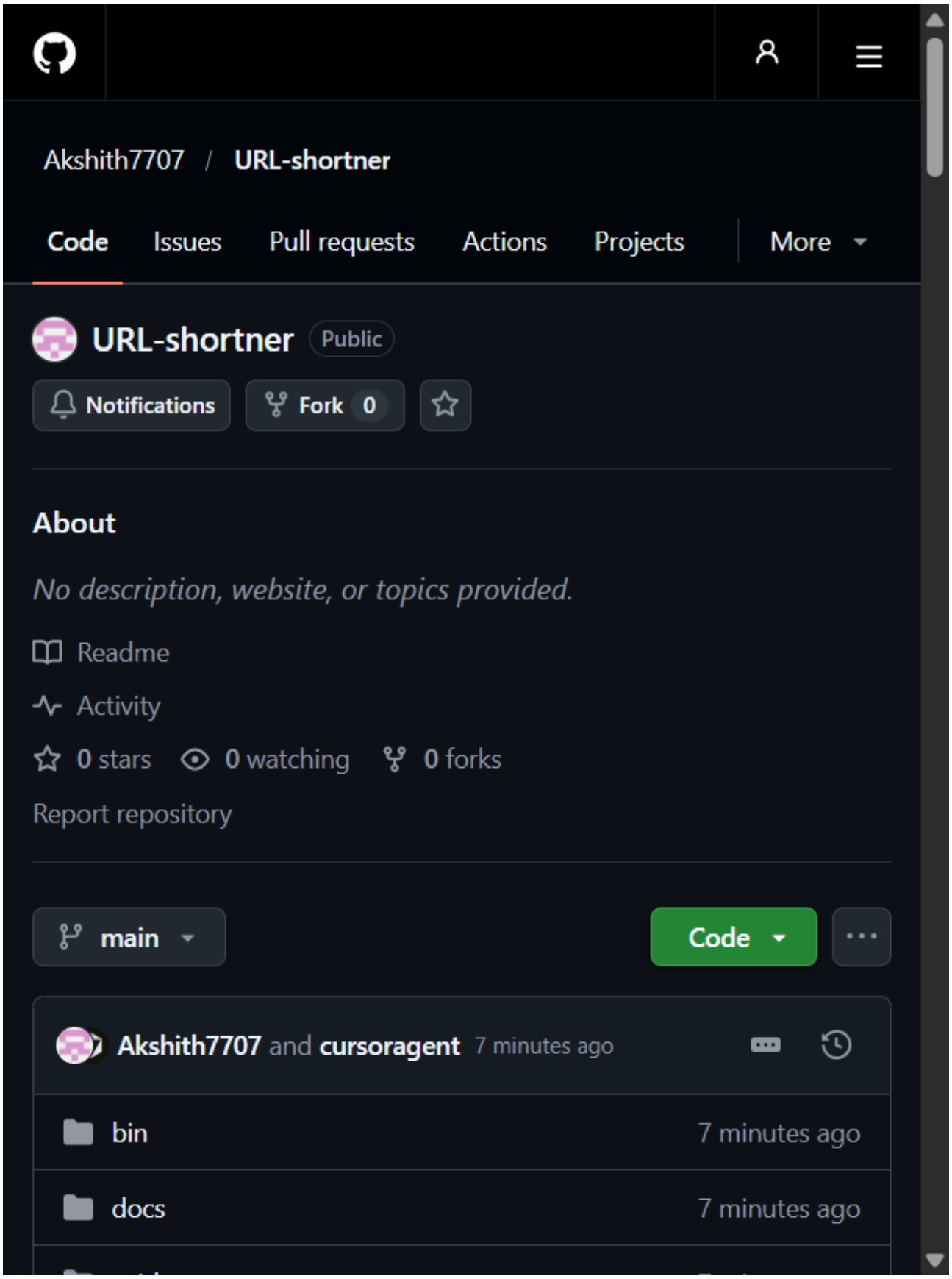


Figure 5. GitHub repository Akshith7707/URL-shortner (main branch) containing project source.

12. How to Run Locally (Zero AWS Cost)

Prerequisites: Node.js, npm, Java (for DynamoDB Local JAR) or Docker Desktop.

- cd into the project root
- npm install
- npm run local:setup:jar (or npm run local:setup if Docker engine is running)
- npm run local:api (serves http://127.0.0.1:3000)
- .\scripts\smoke-test.ps1 -ApiUrl http://127.0.0.1:3000

Detailed instructions: docs/LOCAL.md

13. Cloud Deployment Notes (Future)

The CDK stack is ready for deployment to ap-south-1. Deployment was deferred to avoid AWS charges and because authentication was available only as the AWS root user. Root must not be used for deployments.

- Create a non-root IAM user/role with least privilege
- Configure AWS CLI with that identity
- Approve and run: npx cdk bootstrap (once per account/region if needed)
- Approve and run: npx cdk deploy
- Re-run smoke tests against the ApiUrl stack output

npm run deploy / destroy are blocked in package.json during zero-cost mode.

14. Security Considerations

- No secrets committed to Git
- URL validation rejects localhost and link-local/metadata hosts
- Least-privilege DynamoDB actions per function
- CORS open (*) suitable for demo only; restrict origins for production
- Conditional writes prevent short-code overwrite collisions

15. Results and Conclusion

The project delivers a complete serverless URL shortener design and implementation: infrastructure as code, Lambda application logic, DynamoDB data model, IAM least privilege, monitoring design, documentation, automated tests, and verified local end-to-end behavior with screenshots. The same codebase can be deployed to AWS when a non-root identity and budget approval are available.

Declaration

I, Akshith Reddy Ponna (Roll No: 2451-23-751-028), submit this project report for the Cloud Infrastructure and DevOps Engineer domain. The work presented is part of my project submission URL Shortener using Lambda + DynamoDB.

Student Signature: _____

Date: 31 / 10 / 2026